

Why Good Docstrings Are Not Good Enough

Five well-documented Python tools, fifteen ordinary requests, and the nine wrong answers nobody would have noticed.

9 of 15

requests got the
wrong arguments

0

of those nine
showed an error

15 of 15

correct after the
rewrite

Prepared for Sourav Bhattacharya

Created Sunday, 06 September 2026 at 13:52 (PDT)

Method 5 tools, asked 3 different ways each, tested twice

Code delivered with this document, runnable as-is

Contents

1. The short version	3
2. The five tools we tested	4
3. How we tested	5
Why three phrasings?	5
4. The scoreboard	6
5. The nine mistakes, one at a time	7
5.1 The meeting booked at the wrong hour (2 mistakes)	7
5.2 A company name where an ID belongs	8
5.3 A status word that was not on the list	9
5.4 Two currencies swapped	10
5.5 A text message sent to an email address	11
5.6 A priority level that does not exist	12
5.7 A picture size the tool cannot do (2 mistakes)	13
6. Six rules that came out of this	14
7. Adding input checks	15
A real check, start to finish	16
Every check, and what it says	17
8. The retest	19
9. What the input checks cannot catch	20
10. A checklist for your next tool	20
11. References	21
Appendix A — the finished tools	22
Appendix B — how to run this yourself	31

1. The short version

We wrote five ordinary Python tools. Each one had a careful docstring: every parameter had a clear name, a stated type, and an example. Then we gave each tool to an AI model and asked for the same thing three different ways, in plain English, and watched which arguments it filled in.

Out of 15 requests, the model got 6 right and 9 wrong.

That is not the surprising part. This is: **not one of the nine wrong calls produced an error.** The tool took the bad argument, did something with it, and returned a normal-looking result. The model then told the user the job was done.

Here is what that looks like in practice:

- A meeting booked for 3pm was actually booked for 8am. The confirmation still said 3pm.
- A search for a customer's orders returned an empty list. It read like “this customer has no orders”, not like “your search was broken”.
- A text message was addressed to an email address and handed to the SMS system. Nobody received it.

After we rewrote the nine docstrings and added input checks, the same 15 requests all came back correct. But the two fixes do different jobs, and it is worth knowing which one is doing the work.

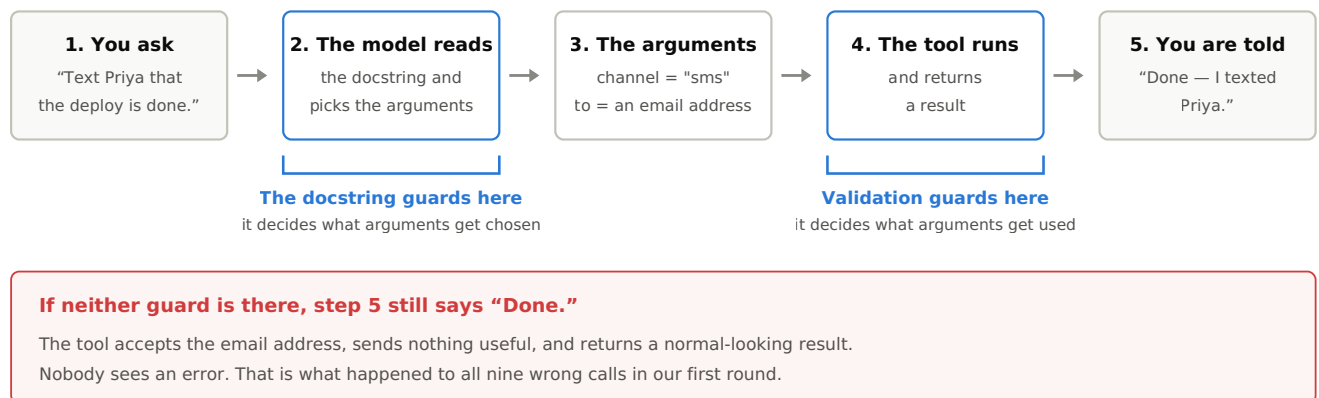


Figure 1. A tool call passes two gates. The docstring decides what arguments get picked. Validation decides what arguments get used. If a value looks fine but means the wrong thing, only the first gate can stop it.

■ right arguments ❌ wrong arguments

Before (original docstrings)



After (rewritten + validation)



15 requests in total: 5 tools, asked 3 different ways each.

Figure 2. The same 15 requests, before and after the rewrite.

2. The five tools we tested

They are the kind of tools any internal assistant might have. Each one was chosen because it makes a different kind of mistake easy to make.

TOOL	WHAT IT DOES	WHAT MAKES IT EASY TO GET WRONG
schedule_meeting	Books a meeting.	A time with no timezone looks completely fine and is completely ambiguous.
search_orders	Searches orders.	It only accepts five exact status words, and users never use those words.
convert_currency	Converts money.	Two parameters hold the same kind of value, so they are easy to swap.
send_notification	Sends one message.	The address format changes depending on the channel you pick.
resize_image	Resizes a picture.	People ask for “half the size”, but the tool only understands pixels.

Here are the five, as we first wrote them:

```
schedule_meeting(attendee_emails, start_datetime_iso, duration_minutes, room_name)

search_orders(customer_id, status, created_after, created_before, max_results)

convert_currency(amount, source_currency, target_currency, rate_date)

send_notification(recipient_handle, channel, message_body, priority)

resize_image(source_path, target_path, width_px, height_px,
              preserve_aspect_ratio, jpeg_quality)
```

Read that list and nothing looks wrong. Every name says what it holds. There is no `data`, no `options`, no `flag`. That is the whole point of this report: good names are not enough on their own.

3. How we tested

For each request, the model was shown two things and nothing else:

1. The tool's name, its parameters, and its docstring.
2. A short block of background facts, the same one every time.

The background facts

Today is Sunday, 6 September 2026. The user is in Los Angeles (UTC−07:00 today).

People: Priya Raman, Marco Silva, Dana Wu — all at @example.com.

Customer: Acme Robotics, account CUST-40912.

Before each test we wrote down the arguments a correct answer would need. Then we compared what the model actually sent, field by field, and ran the tool with it.

A request counts as wrong if any single argument is wrong — whether or not the tool complained. That distinction matters, so it has two names:

- **A loud mistake** is one the tool rejects. Annoying, but at least somebody finds out.
- **A quiet mistake** is one the tool accepts. It returns a sensible-looking answer built on the wrong input, and nobody finds out.

All nine mistakes in our first round were quiet ones. That is not bad luck. A plain Python function only rejects the values that happen to crash it, and most wrong values do not crash anything.

Why three phrasings?

Because people do not ask for things the same way twice. For each tool, one phrasing happened to use the same words the tool uses, and one or two did not. Almost every request that passed was the one where the user happened to speak the tool's language. Every request that failed was one where the docstring had to do the translating — and could not.

4. The scoreboard

TOOL	WHAT WE ASKED	RESULT
<code>schedule_meeting</code>	Book a 30 minute sync with Priya and Marco tomorrow at 3pm.	wrong, no error
<code>schedule_meeting</code>	Set up an hour-long meeting with Dana in the Bridgewater room on Sept 9 at 10am.	wrong, no error
<code>schedule_meeting</code>	Book 15 minutes with Dana in Bridgewater at 9am Pacific on the 8th.	correct
<code>search_orders</code>	Show me all of Acme's cancelled orders from last week.	wrong, no error
<code>search_orders</code>	What orders have shipped for CUST-40912 in the past 3 days?	correct
<code>search_orders</code>	List CUST-40912's completed orders this month — just the top 5.	wrong, no error
<code>convert_currency</code>	Convert 500 US dollars to euros.	correct
<code>convert_currency</code>	What's the euro price of an \$89 subscription?	wrong, no error
<code>convert_currency</code>	What was \$1,200 worth in pounds on the first of August?	correct
<code>send_notification</code>	Text Priya that the deploy is done.	wrong, no error
<code>send_notification</code>	Send a high-priority Slack message to @dana about the outage.	correct
<code>send_notification</code>	Email marco.silva@example.com the release notes and mark it urgent.	wrong, no error
<code>resize_image</code>	Resize logo.png to 800 pixels wide.	wrong, no error
<code>resize_image</code>	Shrink hero.jpg by half and save it as hero_small.jpg.	wrong, no error
<code>resize_image</code>	Make banner.png exactly 1200x630 for the OG image, high quality.	correct

Three things stand out.

One parameter never failed once: `duration_minutes`. It was filled in correctly for “30 minute”, “hour-long” and “15 minutes”. Nobody ever sent 1 to mean one hour. The reason is simple: the unit is in the name, right where the value gets typed. The docstring did not have to say a word about it.

Mistakes cluster on values the model cannot work out from the sentence. A timezone that lives in someone's profile. An account ID that lives in another system. The size of a file on disk. When the model cannot work out the answer, it does not stop and ask. It writes down its best guess, neatly formatted.

Listing the allowed values is not the same as saying “these are the only ones”. Both of our list-based parameters had their values written out in the docstring. The model still invented new ones.

5. The nine mistakes, one at a time

Each one is told the same way: what was asked, what the model sent, what happened, why, and the one change that fixed it.

5.1 The meeting booked at the wrong hour (2 mistakes)

WHAT YOU ASKED FOR

“Book a 30 minute sync with Priya and Marco tomorrow at 3pm.”

WHAT THE MODEL SENT

```
start_datetime_iso = "2026-09-07T15:00:00"
```

WHAT HAPPENED

The meeting was booked. The confirmation said 15:00. The actual slot was 8am Pacific — seven hours early.

WHY

The time has no timezone on the end. The server assumes a time with no timezone is UTC. The user meant 3pm in Los Angeles. Nothing in the docstring said what happens when the timezone is missing, so the model had no reason to think it mattered — and the example in the docstring was missing it too.

THE FIX

DOCSTRING BEFORE

```
start_datetime_iso : str
    An ISO 8601 datetime string,
    e.g. "2026-09-07T15:00:00".
```

DOCSTRING AFTER

```
start_datetime_with_utc_offset : str
    An ISO 8601 datetime. It MUST end
    with a timezone offset.
    Good: "2026-09-07T15:00:00-07:00"
    Bad:  "2026-09-07T15:00:00"
```

Two changes. The name now says what has to be there. And there is a **Bad** example showing the exact mistake, not just a good one. **The model copies your example.** If your example is missing something, so is the value you get back.

The same mistake happened again on the next request, phrased differently. It only stopped when the docstring changed.

5.2 A company name where an ID belongs

WHAT YOU ASKED FOR

“Show me all of Acme's cancelled orders from last week.”

WHAT THE MODEL SENT

```
customer_id = "Acme Robotics"
```

WHAT HAPPENED

The search ran and found nothing. To the user it looked like Acme had no cancelled orders.

WHY

`customer_id` needs an account ID like `CUST-40912`. The user said “Acme”. The model had no way to turn a company name into an ID, and the docstring never mentioned that there was a way. Faced with a required box and nothing to put in it, the model wrote down the closest thing it had.

THE FIX

DOCSTRING BEFORE

```
customer_id : str
    The customer's account
    identifier, e.g. "CUST-40912".
```

DOCSTRING AFTER

```
customer_account_id : str
    An account ID: "CUST-" plus five
    digits, e.g. "CUST-40912".
    A company name is NOT an ID. If the
    user says "Acme", call
    lookup_customer_id("Acme") first.
    Do not guess or make one up.
```

Telling the model “don't guess” only works if you also tell it where to go instead. Give it somewhere to go and it goes there. Give it a rule with no alternative and it just guesses differently.

5.3 A status word that was not on the list

WHAT YOU ASKED FOR

"List CUST-40912's completed orders this month."

WHAT THE MODEL SENT

```
status = "completed"
```

WHAT HAPPENED

Zero results. Again, it looked like a real answer.

WHY

The tool only accepts five words: `pending`, `paid`, `shipped`, `delivered`, `cancelled`. The word for "completed" is `delivered`. The docstring listed all five, but never said the list was the whole list — so the model added a sixth word that seemed to fit.

THE FIX

DOCSTRING BEFORE

```
status : str
    Filter to orders in this state.
    One of: pending, paid, shipped,
    delivered, cancelled.
```

DOCSTRING AFTER

```
order_status : str
    ONLY these five words are allowed:
    "pending"    placed, not paid yet
    "paid"       paid, not sent yet
    "shipped"    on its way
    "delivered"  arrived (=
"completed")
    "cancelled"  cancelled or refunded
    Users say different words.
    Translate:
    completed / done -> "delivered"
    refunded         -> "cancelled"
    in transit       -> "shipped"
```

Two things were added. First, **ONLY these five are allowed** — which is different from just listing five. Second, a translation table from the words people actually say to the words the tool accepts. Do that translation in the docstring, so the model does not have to improvise it.

5.4 Two currencies swapped

WHAT YOU ASKED FOR

“What's the euro price of an \$89 subscription?”

WHAT THE MODEL SENT

```
source_currency = "EUR"  
target_currency = "USD"
```

WHAT HAPPENED

It converted 89 euros into dollars instead of 89 dollars into euros. The answer came back as €96.74. The right answer was €81.88.

WHY

The sentence mentions euros first and dollars second, but the number (89) is in dollars. To fill in `source_currency`, the model has to work out a direction from the sentence, and the nearest currency word was the wrong one. The docstring said “the currency the money is currently in”, which is correct and did not help.

THE FIX

DOCSTRING BEFORE

```
source_currency : str  
    The currency the money is  
    currently in.  
target_currency : str  
    The currency to convert into.
```

DOCSTRING AFTER

```
amount_currency_code : str  
    The currency `amount` is written in.  
result_currency_code : str  
    The currency the answer comes back  
    in.
```

Ask: "what unit is the NUMBER in?"
That is `amount_currency_code` – no matter
where the words appear in the sentence.

The old names describe a *direction*, which the reader has to reconstruct. The new names describe a *fact*: one is the currency of the number, the other is the currency of the answer. A sentence written backwards can no longer flip them.

5.5 A text message sent to an email address

WHAT YOU ASKED FOR

"Text Priya that the deploy is done."

WHAT THE MODEL SENT

```
channel = "sms"
recipient_handle = "priya.raman@example.com"
```

WHAT HAPPENED

The tool accepted it and reported success. Priya received nothing.

WHY

This was the whole docstring for that parameter: *"Who to notify. The format depends on channel."* It then never said what any of the formats were. The model only had Priya's email address, so that is what it used. A sentence that points at another parameter without answering the question is worse than no sentence, because it looks like documentation.

THE FIX

DOCSTRING BEFORE

```
recipient_handle : str
    Who to notify. The format
    depends on `channel`.
```

DOCSTRING AFTER

```
recipient_address : str
    The address, in the format that
    `channel` requires. They are NOT
    interchangeable:
        "email" -> priya@example.com
        "sms"   -> +14155550142
        "slack" -> @priya
        "push"  -> a 32+ char device token
    You usually only have someone's
    email.
    That does not work on other
    channels:
        call lookup_contact(person,
        channel).
```

If a format depends on another parameter, write out every case. One line each, with a real example. It costs four lines and removes the guesswork completely.

5.6 A priority level that does not exist

WHAT YOU ASKED FOR

"Email marco.silva@example.com the release notes and mark it urgent."

WHAT THE MODEL SENT

```
priority = "urgent"
```

WHAT HAPPENED

Accepted, and passed straight through. Whether it actually counted as urgent depended on what the delivery system did with an unknown word.

WHY

The allowed values are `low`, `normal` and `high`. The user said "urgent", and the model kept the user's word rather than translating it. That is a reasonable instinct. Nothing told it the list was closed, and nothing told it where "urgent" should go.

THE FIX

DOCSTRING BEFORE

```
priority : str
    Delivery priority. One of:
    low, normal, high.
```

DOCSTRING AFTER

```
priority : str
    ONLY these three are allowed:
    "low", "normal", "high".
    Translate what the user says:
    urgent / ASAP / critical -> "high"
    medium / standard       ->
    "normal"
    FYI / no rush           -> "low"
    There is nothing above "high".
```

Same pattern as the status list. Mark the set as closed, then hand over the translation instead of hoping the model guesses it.

5.7 A picture size the tool cannot do (2 mistakes)

WHAT YOU ASKED FOR

"Shrink hero.jpg by half and save it as hero_small.jpg."

WHAT THE MODEL SENT

```
width_px = 50
height_px = 50
```

WHAT HAPPENED

It produced a 50×50 pixel thumbnail instead of an image half the original size.

WHY

The tool only understands exact pixels, and the model does not know how big the picture currently is. It sent 50 meaning "50 percent". The tool read it as "50 pixels". The docstring never said the numbers are absolute pixels, and never said what to do when someone asks for something relative.

THE FIX

DOCSTRING BEFORE

```
width_px : int
    Desired width in pixels.
height_px : int
    Desired height in pixels.
```

DOCSTRING AFTER

This tool understands EXACT PIXELS ONLY. It cannot scale by a percentage, and it does not know the picture's current size.

For "half the size" or "25% smaller": call `get_image_info(source_path)` first, do the maths, then pass real pixels.

Never send 50 to mean "50%".
Never send 0.5 to mean "half".

The most useful thing this docstring now does is say what the tool *cannot* do. On the retest the model called `get_image_info`, read 1920×1080, and sent 960×540. The docstring turned a guess into a lookup.

The second mistake with this tool was smaller but the same shape. Asked to "resize logo.png to 800 pixels wide", the model sent `width_px=800` and `height_px=800`, because `height_px` was a required parameter and it had to put something there. The fix was to make it optional, and to say so:

```
height_px : int or None
    The height in pixels, or None to work it out from the width.
    "800 pixels wide" -> width_px=800, height_px=None
    Do NOT repeat the width as the height. width_px=800 with
    height_px=800 means "fit inside an 800x800 box", which is a
    different request.
```

If a parameter is required but the caller has no way to know its value, that is a problem with the tool, not with the caller.

6. Six rules that came out of this

Every one of these came from a mistake above, not from taste.

Rule 1

Put the unit or the format in the parameter name.

`duration_minutes` never failed once, because the unit is sitting right there when the value gets written. The same idea fixed the `datetime ( start_datetime_with_utc_offset )` and the currency swap (`amount_currency_code`). If a name needs the docstring to be clear, rename it before you write another word.

Rule 2

Show a bad example, not just a good one.

The model copies your example. Our `datetime` example had no timezone, so neither did the value we got back. Adding one line — *Bad: "2026-09-07T15:00:00" (no offset)* — fixed both timezone mistakes.

Rule 3

Say “these are the only allowed values”, and translate the words people actually use.

Listing five values is not the same as ruling out a sixth. Mark the list closed, then add the mapping: *completed* → *delivered*, *urgent* → *high*. You know that translation. Write it down instead of hoping it gets guessed.

Rule 4

If a format depends on another parameter, list every case.

“The format depends on `channel` ” tells the reader nothing. Four lines, one per channel, each with a real example, tells them everything.

Rule 5

If the caller cannot know a value, say where to get it — then say “do not guess”.

A required box with no way to fill it does not produce a question. It produces a confident invention. Naming the lookup tool is what turns the guess into a lookup; the ban on guessing on its own just produces a different guess.

Rule 6

Say what the tool cannot do.

Docstrings describe what a tool can do and go quiet about the rest. Ask for something outside that, and the request gets squeezed into whatever shape is available. Writing down the limit prevents it.

The rule that did not come out of the data

“Write more detail.” Our first docstrings were already detailed. Every failing parameter had a type and most had an example. What was missing was never volume — it was one specific sentence closing one specific hole. And in three of the nine cases the real fix was **renaming the parameter**, which no amount of extra prose replaces.

7. Adding input checks

A better docstring makes a wrong call less likely. It does not make one impossible, and it does nothing at all when a different model, an older model, or a person calls your tool.

So every tool now checks its inputs before doing anything, and raises `ValueError` when something is wrong. Every message is built the same way, from one small helper:

```
def fail(param, got, problem, fix):  
    """Raise a ValueError that tells the caller how to fix it."""  
    raise ValueError(f"{param}: {problem} Received {got!r}. {fix}")
```

Used like this:

```
fail("priority", "urgent",  
     "is not an allowed value.",  
     "Allowed values: low, normal, high. "  
     "'urgent' most likely means 'high' – pass that instead.")
```

Which produces this message:

```
priority: is not an allowed value. Received 'urgent'. Allowed values: low, normal, high.  
'urgent' most likely means 'high' – pass that instead.
```



Figure 3. Every error message has the same three parts. The third one is the one usually missing: a caller told what is wrong, but not what to do instead, tends to retry with the same kind of mistake.

A real check, start to finish

Here is the whole check for the status word, including the “did you mean” hint:

```
ALLOWED = ("pending", "paid", "shipped", "delivered", "cancelled")

SYNONYMS = {"completed": "delivered", "done": "delivered",
            "refunded": "cancelled", "in transit": "shipped"}

def check_status(value):
    word = value.strip().lower()

    if word in ALLOWED:
        return word                                # all good

    if word in SYNONYMS:                           # a near miss – name the right word
        fail("order_status", value,
            f"'{value}' is not an allowed value.",
            f"Allowed values: {' '.join(ALLOWED)}. "
            f"'{word}' most likely means '{SYNONYMS[word]}'.")

    fail("order_status", value,                     # not close to anything
        f"'{value}' is not an allowed value.",
        f"Allowed values: {' '.join(ALLOWED)}. This list is closed – "
        f"if none of them fit, tell the user instead of guessing.")
```

Four things about that are worth copying into your own tools:

- **The message repeats what was sent.** The caller does not have to remember what it passed — which matters a lot when the arguments were generated rather than typed.
- **A near miss gets named.** “completed most likely means delivered” is a fix. “Invalid status” is a complaint.
- **A check that spans two parameters gives advice about both.** An email address on the SMS channel does not just say “invalid”; it says which lookup to call.
- **The checks run first.** A rejected call has changed nothing.

Every check, and what it says

We wrote 18 deliberately broken calls — the kind of mistake a model actually makes. All 18 now raise `ValueError`.

TOOL	THE BROKEN CALL	RESULT
<code>schedule_meeting</code>	one email address instead of a list of them	caught
<code>schedule_meeting</code>	people's names instead of email addresses	caught
<code>schedule_meeting</code>	a time with no timezone on the end	caught
<code>schedule_meeting</code>	1 sent to mean "one hour"	caught
<code>search_orders</code>	a company name where an account ID belongs	caught
<code>search_orders</code>	the made-up status "completed"	caught
<code>search_orders</code>	"last week" instead of a real date	caught
<code>search_orders</code>	a date range that covers no days at all	caught
<code>convert_currency</code>	the text "1,200" instead of the number 1200	caught
<code>convert_currency</code>	"dollars" instead of "USD"	caught
<code>convert_currency</code>	the same currency on both sides	caught
<code>send_notification</code>	an email address sent by SMS	caught
<code>send_notification</code>	the priority "urgent", which is not on the list	caught
<code>send_notification</code>	the channel "text" instead of "sms"	caught
<code>resize_image</code>	0.5 sent to mean "half the size"	caught
<code>resize_image</code>	no width and no height given	caught
<code>resize_image</code>	an exact size with only one side given	caught
<code>resize_image</code>	the text "800px" instead of the number 800	caught

Four of the messages, exactly as they come out

1 sent to mean "one hour"

```
duration_minutes: is outside the allowed range of 5-480 minutes. Received 1. If the user really wants a longer block, book multiple events.
```

a company name where an account ID belongs

```
customer_account_id: is not a valid account id (expected CUST- followed by five digits, e.g. 'CUST-40912'). Received 'Acme Robotics'. If this is a company or person name, resolve it with `lookup_customer_id` first, or pass None to search all customers. Do not fabricate an id.
```

an email address sent by SMS

recipient_address: is not valid for channel='sms', which requires an E.164 phone number starting with '+'. Received 'priya.raman@example.com'. Example: '+14155550142'. You passed an email address to channel='sms'. Call lookup_contact(person, 'sms') to get their sms address, or send it by email instead.

0.5 sent to mean “half the size”

width_px: looks like a scale factor rather than a pixel count. Received 0.5. This tool takes absolute pixels only. Call `get\_image\_info(source\_path)`, multiply the real dimension yourself, and pass the result, e.g. 960.

8. The retest

Same 15 requests. Same background facts. New docstrings. **All 15 correct.**

TOOL	WHAT WE ASKED	BEFORE	AFTER
<code>schedule_meeting</code>	Book a 30 minute sync with Priya and Marco tomorrow at 3pm.	wrong	correct
<code>schedule_meeting</code>	Set up an hour-long meeting with Dana in the Bridgewater room on Sept 9 at 10am.	wrong	correct
<code>schedule_meeting</code>	Book 15 minutes with Dana in Bridgewater at 9am Pacific on the 8th.	correct	correct
<code>search_orders</code>	Show me all of Acme's cancelled orders from last week.	wrong	correct
<code>search_orders</code>	What orders have shipped for CUST-40912 in the past 3 days?	correct	correct
<code>search_orders</code>	List CUST-40912's completed orders this month — just the top 5.	wrong	correct
<code>convert_currency</code>	Convert 500 US dollars to euros.	correct	correct
<code>convert_currency</code>	What's the euro price of an \$89 subscription?	wrong	correct
<code>convert_currency</code>	What was \$1,200 worth in pounds on the first of August?	correct	correct
<code>send_notification</code>	Text Priya that the deploy is done.	wrong	correct
<code>send_notification</code>	Send a high-priority Slack message to @dana about the outage.	correct	correct
<code>send_notification</code>	Email marco.silva@example.com the release notes and mark it urgent.	wrong	correct
<code>resize_image</code>	Resize logo.png to 800 pixels wide.	wrong	correct
<code>resize_image</code>	Shrink hero.jpg by half and save it as hero_small.jpg.	wrong	correct
<code>resize_image</code>	Make banner.png exactly 1200x630 for the OG image, high quality.	correct	correct

Two of the fixes changed what the model *did*, not just what it typed.

- For “shrink hero.jpg by half”, it stopped guessing and called `get_image_info` first.
- For “Acme's orders”, it stopped inventing an ID and called `lookup_customer_id`.

Both worked for the same reason: the docstring gave the model somewhere else to go.

9. What the input checks cannot catch

We then asked a separate question. If a model still makes one of the original nine mistakes, do the new checks stop it? For six of them, yes. For three, no.

Validation catches these	6 of 9	Only the docstring catches these	3 of 9
The value is the wrong shape, so a check can spot it.		The value looks perfectly fine. It just means the wrong thing.	
• Wrong timezone • Company name, not an ID • A status word that isn't allowed • Email address on the SMS channel • Priority "urgent"		• Two currencies swapped • Height copied from the width • 50 meaning "50 percent"	

Figure 4. The three on the right are the ones to worry about. Every one of them is a perfectly valid value that simply means the wrong thing.

Look at why those three get through:

- `height_px = 800` is a perfectly good pixel count. It is just not the one the user asked for.
- `width_px = 50` is a perfectly good pixel count. The model meant 50 percent.
- `amount_currency_code = "EUR"` is a real, supported currency code. It is just the wrong side of the conversion.

No check can tell those apart from correct calls, because the thing that would tell them apart — what the user actually meant — is not in the arguments at all.

That leaves exactly one place to catch them: the docstring, before the arguments are chosen. This is the opposite of the usual instinct, where runtime checks are the real defence and documentation is what you write afterwards if there is time.

The one-line version

Input checks catch calls that are **malformed**. Only the docstring catches calls that are **wrong**. You need both, and one cannot replace the other.

10. A checklist for your next tool

Go through this once per parameter.

1. **Does the name say the unit, the format, or the frame of reference?** If it needs the docstring to make sense, rename it first.

2. **Is your example complete?** Whatever your example is missing, the value you get back will be missing too.
3. **Did you show a bad example as well as a good one?**
4. **If there is a fixed list of values, did you say it is the only list, and did you translate everyday words onto it?**
5. **If the format depends on another parameter, did you write out every case?**
6. **Can the caller actually find this value?** If not, name the tool that provides it and say “do not guess”.
7. **Did you say what the tool cannot do?**
8. **If a date range has an edge, did you say whether it is included, and show the awkward case?**
9. **Does a bad value raise `ValueError` naming the parameter, repeating what arrived, and stating the fix?**
10. **Did you test it with three phrasings you did not write while writing the docstring?**
The one that passes is usually the one where you accidentally used your own tool's words.
The other two are the real test.

11. References

1. The code and results from this study, delivered alongside this document: `tools_v1.py` (the original tools), `tools_v2.py` (the fixed tools), `cases.py` and `cases_v2.py` (the 15 requests and the arguments the model sent), `run_round1.py` and `run_round2.py` (the test runners), and the raw results as JSON.
2. ISO 8601 — the international standard for writing dates and times. This is where the timezone offset requirement comes from.
3. ISO 4217 — the standard three-letter currency codes (USD, EUR, GBP).
4. ITU-T Recommendation E.164 — the international phone number format used for the SMS channel (a `+`, then country code, then digits).
5. PEP 257, *Docstring Conventions*, and the NumPy docstring style — the docstring layout used throughout.
6. Python documentation, *Built-in Exceptions* — `ValueError` is defined as the error for a value of the right type but the wrong content, which is exactly what every check here is testing.

Appendix A — the finished tools

This is `tools_v2.py`: the rewritten docstrings and all the input checks in one file. The original version is in the accompanying `tools_v1.py` if you want to compare them side by side.

```
"""
tools_v2.py – Round 2 toolkit.

Every docstring that produced a wrong call in Round 1 has been rewritten,
and every tool now validates its inputs and raises ValueError with a message
that states what arrived, what is allowed, and what to do instead.

Docstring rules applied after Round 1:
R1. Name the parameter after what the value IS, never after a direction
    or a relationship ("amount_currency_code", not "source_currency").
R2. Closed sets are labelled CLOSED SET and listed as exact literals, with
    a synonym table mapping the words users actually say onto them.
R3. Every format is spelled out as a pattern plus an example, per channel
    or per case – never "the format depends on X".
R4. Boundary semantics (inclusive / exclusive) are stated, with a worked
    example for the common phrasing.
R5. If a value cannot be derived from the conversation, say which tool to
    call for it and say "do not guess".
R6. State what the tool cannot do, so the model stops instead of coercing.
"""

import re
from datetime import datetime

_ORDER_STATUSES = ("pending", "paid", "shipped", "delivered", "cancelled")
_STATUS_SYNONYMS = {"completed": "delivered", "complete": "delivered",
                    "fulfilled": "delivered", "done": "delivered",
                    "refunded": "cancelled", "void": "cancelled",
                    "voided": "cancelled", "in transit": "shipped",
                    "dispatched": "shipped", "unpaid": "pending",
                    "awaiting payment": "pending"}
_PRIORITIES = ("low", "normal", "high")
_PRIORITY_SYNONYMS = {"urgent": "high", "high": "high", "asap": "high", "critical": "high",
                     "p0": "high", "emergency": "high", "important": "high",
                     "medium": "normal", "standard": "normal",
                     "fyi": "low", "whenever": "low"}
_CHANNELS = ("email", "sms", "slack", "push")
_FX = {"USD": 1.0, "EUR": 0.92, "GBP": 0.79, "JPY": 156.4, "INR": 83.2}

_RE_EMAIL = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
_RE_E164 = re.compile(r"^[+][1-9]\d{7,14}$")
_RE_SLACK = re.compile(r"^(@[a-z0-9_-]+|([UW][A-Z0-9]{6,}))$")
_RE_PUSH = re.compile(r"^[A-Za-z0-9_-]{32}$")
_RE_CUSTOMER = re.compile(r"^[CUST-\d{5}]$")
_RE_DATE = re.compile(r"^[d{4}-d{2}-d{2}]$")

# -----
# Shared validation helpers
# -----
def _fail(param, received, problem, fix):
    raise ValueError(
        f"{param}: {problem} Received {received!r}. {fix}")

def _closed_set(param, value, allowed, synonyms):
    if not isinstance(value, str):
        _fail(param, value, f"expected a string from a closed set of "
              f"{len(allowed)} values.",
              f"Allowed values: {' '.join(allowed)}")
    key = value.strip().lower()
```

```

if key in allowed:
    return key
if key in synonyms:
    _fail(param, value,
          f"'{value}' is not one of the accepted values.",
          f"Allowed values: {'', '.join(allowed)}. "
          f"'{key}' most likely means '{synonyms[key]}' – pass that instead.")
_fail(param, value, f"'{value}' is not one of the accepted values.",
      f"Allowed values: {'', '.join(allowed)}. This set is closed; do not "
      f"invent a synonym. If none of these fit, tell the user rather than "
      f"guessing.")

```

```

# -----
# Tool 1
# -----

```

```

def schedule_meeting(attendee_emails, start_datetime_with_utc_offset,
                     duration_minutes, room_name=None):
    """Book a meeting on the shared team calendar.

```

Parameters

attendee_emails : list[str]

A LIST of full email addresses, one per invitee, even when there is only one invitee. Each item must match name@domain.tld.

Correct: ["priya.raman@example.com", "marco.silva@example.com"]

Rejected: "priya.raman@example.com" (string, not a list)

Rejected: ["Priya", "Marco"] (display names, not addresses)

If you only have a person's name, resolve it with ``lookup_person`` first. Do not guess an address.

start_datetime_with_utc_offset : str

The instant the meeting starts, as an ISO 8601 datetime that MUST carry an explicit UTC offset or a trailing "Z". A naive string is rejected – there is no default timezone.

Correct: "2026-09-07T15:00:00-07:00" (3pm in America/Los Angeles)

Correct: "2026-09-07T22:00:00Z" (the same instant, in UTC)

Rejected: "2026-09-07T15:00:00" (no offset – ambiguous)

Rejected: "tomorrow at 3pm" (not ISO 8601)

When the user says a wall-clock time ("3pm"), it is a wall-clock time in THEIR timezone: take their current UTC offset and append it. Do not relabel local time as UTC.

duration_minutes : int

Meeting length in whole MINUTES. Integer, 5 to 480 inclusive.

"half an hour" -> 30, "an hour" -> 60, "90 minutes" -> 90.

room_name : str or None

Exact name of a physical room, e.g. "Bridgewater". Pass None for a video-only meeting. Do not pass "" or "none".

Raises

ValueError

If any argument is missing, mistyped, or outside the stated range.

"""

```

if isinstance(attendee_emails, str):

```

```

    _fail("attendee_emails", attendee_emails,

```

```

        "expected a list of email addresses, got a single string.",

```

```

        "Wrap it in a list, e.g. ['priya.raman@example.com'].")

```

```

if not isinstance(attendee_emails, (list, tuple)) or not attendee_emails:

```

```

    _fail("attendee_emails", attendee_emails,

```

```

        "expected a non-empty list of email addresses.",

```

```

        "Example: ['priya.raman@example.com'].")

```

```

bad = [a for a in attendee_emails

```

```

      if not (isinstance(a, str) and _RE_EMAIL.match(a))]

```

```

if bad:

```

```

    _fail("attendee_emails", bad,

```

```

        "every item must be a full email address (name@domain.tld).",

```

```

        "These entries look like display names. Resolve them with "

```

```

        "`lookup_person` first; do not guess an address.")

```

```

if not isinstance(start_datetime_with_utc_offset, str):

```

```

    _fail("start_datetime_with_utc_offset", start_datetime_with_utc_offset,

```

```

        "expected an ISO 8601 string.",
        "Example: '2026-09-07T15:00:00-07:00'."))
try:
    start = datetime.fromisoformat(
        start_datetime_with_utc_offset.replace("Z", "+00:00"))
except ValueError:
    _fail("start_datetime_with_utc_offset", start_datetime_with_utc_offset,
        "is not a parseable ISO 8601 datetime.",
        "Use YYYY-MM-DDTHH:MM:SS followed by a UTC offset, "
        "e.g. '2026-09-07T15:00:00-07:00'.")
if start.tzinfo is None:
    _fail("start_datetime_with_utc_offset", start_datetime_with_utc_offset,
        "has no UTC offset, so the intended instant is ambiguous.",
        "Append the user's current UTC offset (or 'Z' for UTC), "
        "e.g. '2026-09-07T15:00:00-07:00' for 3pm Pacific. There is no "
        "default timezone.")

if isinstance(duration_minutes, bool) or not isinstance(duration_minutes, int):
    _fail("duration_minutes", duration_minutes,
        "expected a whole number of minutes.",
        "Use 60 for one hour, not 1 and not '1h'.")
if not 5 <= duration_minutes <= 480:
    _fail("duration_minutes", duration_minutes,
        "is outside the allowed range of 5-480 minutes.",
        "If the user really wants a longer block, book multiple events.")

if room_name is not None:
    if not isinstance(room_name, str) or not room_name.strip() or \
        room_name.strip().lower() in {"none", "null", "n/a"}:
        _fail("room_name", room_name,
            "expected a real room name or None.",
            "Pass None (not '' or 'none') for a video-only meeting.")

end = start.fromtimestamp(start.timestamp() + duration_minutes * 60,
    start.tzinfo)
return {"event_id": "evt_0001", "starts_at": start.isoformat(),
    "ends_at": end.isoformat(), "invited": list(attendee_emails),
    "room": room_name}

```

```

# -----
# Tool 2
# -----
def search_orders(customer_account_id, order_status, created_on_or_after,
    created_strictly_before, max_results=20):
    """Search the order database.

Parameters
-----
customer_account_id : str or None
    An existing account id matching the pattern CUST-#### (the literal
    text "CUST-" followed by exactly five digits), e.g. "CUST-40912".
    A company name is NOT an account id: if the user says "Acme", call
    `lookup_customer_id("Acme")` first. Do not guess or fabricate an id.
    Pass None to search across all customers.
order_status : str
    CLOSED SET – exactly one of these five literals, lowercase:
        "pending"    order placed, not yet paid
        "paid"       payment captured, not yet dispatched
        "shipped"    handed to the carrier, in transit
        "delivered"  received by the customer (this is "completed")
        "cancelled"  cancelled or refunded before delivery
    Users rarely use these words. Map their wording onto the set:
        completed / complete / fulfilled / done    -> "delivered"
        refunded / void / voided                   -> "cancelled"
        in transit / dispatched                    -> "shipped"
        unpaid / awaiting payment                  -> "pending"
    If the user's meaning maps to none of the five, ask them – do not
    invent a sixth value.
created_on_or_after : str
    INCLUSIVE lower bound, ISO 8601 calendar date "YYYY-MM-DD". An order

```

```

        created at any time on this date is included.
    created_strictly_before : str
        EXCLUSIVE upper bound, ISO 8601 calendar date "YYYY-MM-DD". Orders
        created on this date are NOT included, so to cover a range that ends
        today you must pass TOMORROW's date.
        Worked example – today is 2026-09-06 and the user says "the last
        week", meaning 2026-08-30 through today inclusive:
            created_on_or_after = "2026-08-30"
            created_strictly_before = "2026-09-07" <- tomorrow, not today
        The bound must be strictly after created_on_or_after.
    max_results : int
        Maximum records returned, integer 1-500. Default 20. "top 5" -> 5.

    Raises
    -----
    ValueError
        If the id format, status literal, date format or window is wrong.
    """
    if customer_account_id is not None:
        if not isinstance(customer_account_id, str) or \
            not _RE_CUSTOMER.match(customer_account_id):
            _fail("customer_account_id", customer_account_id,
                "is not a valid account id (expected CUST- followed by five "
                "digits, e.g. 'CUST-40912').",
                "If this is a company or person name, resolve it with "
                "`lookup_customer_id` first, or pass None to search all "
                "customers. Do not fabricate an id.")

    order_status = _closed_set("order_status", order_status,
                                _ORDER_STATUSES, _STATUS_SYNONYMS)

    dates = {}
    for name, raw in (("created_on_or_after", created_on_or_after),
                      ("created_strictly_before", created_strictly_before)):
        if not isinstance(raw, str) or not _RE_DATE.match(raw):
            _fail(name, raw,
                "expected an ISO 8601 calendar date formatted YYYY-MM-DD.",
                "Example: '2026-09-01'. Do not pass a datetime, a relative "
                "phrase such as 'last week', or a localised date.")

        try:
            dates[name] = datetime.fromisoformat(raw).date()
        except ValueError:
            _fail(name, raw, "is not a real calendar date.",
                "Check the month and day, e.g. '2026-09-31' does not exist.")

    if dates["created_strictly_before"] <= dates["created_on_or_after"]:
        _fail("created_strictly_before", created_strictly_before,
            "must be strictly after created_on_or_after "
            f"({created_on_or_after!r}); the window is empty as given.",
            "Remember the upper bound is exclusive: to include today, pass "
            "tomorrow's date.")

    if isinstance(max_results, bool) or not isinstance(max_results, int):
        _fail("max_results", max_results, "expected a whole number.",
            "Pass an integer such as 5 or 20, not a string.")
    if not 1 <= max_results <= 500:
        _fail("max_results", max_results, "is outside the range 1-500.",
            "Page through the results if you need more than 500.")

    return {"query": {"customer_account_id": customer_account_id,
                     "order_status": order_status,
                     "window": [str(dates["created_on_or_after"]),
                                str(dates["created_strictly_before"])],
                     "limit": max_results},
            "results": []}

# -----
# Tool 3
# -----
def convert_currency(amount, amount_currency_code, result_currency_code,
                    rate_date=None):

```

"""Convert a money amount from one currency into another.

The two currency parameters are named after WHAT THEY DESCRIBE, not after a direction, because "from"/"source" invert easily when a sentence names the destination first:

amount_currency_code = the currency `amount` is written in
result_currency_code = the currency the answer comes back in

Read the sentence and ask "what unit is the number in?" – that is amount_currency_code, regardless of word order.

Worked examples

"Convert 500 US dollars to euros."

amount=500.0, amount_currency_code="USD", result_currency_code="EUR"

"What's the euro price of an \$89 subscription?"

the number is 89 dollars, so

amount=89.0, amount_currency_code="USD", result_currency_code="EUR"

(NOT amount_currency_code="EUR" – "euro" appears first in the sentence but describes the ANSWER, not the number.)

"How many dollars is 20000 yen?"

amount=20000.0, amount_currency_code="JPY", result_currency_code="USD"

Parameters

amount : float or int

A positive number, already stripped of symbols and separators.

"\$1,200" -> 1200.0. Never pass a string.

amount_currency_code : str

Uppercase 3-letter ISO 4217 code for the currency `amount` is in.

Supported: USD, EUR, GBP, JPY, INR. Names and symbols are rejected:

"dollars", "\$", "usd " are not codes – normalise to "USD".

result_currency_code : str

Uppercase 3-letter ISO 4217 code for the currency of the answer.

Must differ from amount_currency_code.

rate_date : str or None

ISO 8601 date "YYYY-MM-DD" for a historical rate, or None for the

latest rate. Resolve relative phrases ("the first of August") to a concrete date before calling.

Raises

ValueError

If the amount is not a positive number or a code is unsupported.

"""

if isinstance(amount, str):

_fail("amount", amount, "expected a number, got a string.",

"Strip symbols and separators first: '\$1,200' -> 1200.0.")

if isinstance(amount, bool) or not isinstance(amount, (int, float)):

_fail("amount", amount, "expected a number.", "Example: 500.0.")

if amount <= 0:

_fail("amount", amount, "must be greater than zero.",

"Pass the magnitude; sign is not meaningful for a conversion.")

for name, code in (("amount_currency_code", amount_currency_code),

("result_currency_code", result_currency_code)):

if not isinstance(code, str):

_fail(name, code, "expected a 3-letter ISO 4217 code.",

f"Supported: {' '.join(sorted(_FX))}.")

norm = code.strip().upper()

if norm not in _FX:

hint = ""

guess = {"DOLLAR": "USD", "DOLLARS": "USD", "\$": "USD",

"EURO": "EUR", "EUROS": "EUR", "€": "EUR",

"POUND": "GBP", "POUNDS": "GBP", "£": "GBP",

"YEN": "JPY", "¥": "JPY",

"RUPEE": "INR", "RUPEES": "INR", "₹": "INR"}.get(norm)

if guess:

hint = f" '{code}' is a name or symbol, not a code – use '{guess}'."

_fail(name, code, "is not a supported ISO 4217 currency code.",

```

        f"Supported: {' '.join(sorted(_FX)).hint}")
    if name == "amount_currency_code":
        amount_currency_code = norm
    else:
        result_currency_code = norm

    if amount_currency_code == result_currency_code:
        _fail("result_currency_code", result_currency_code,
            "is the same as amount_currency_code, so there is nothing to "
            "convert.",
            "Re-read the sentence: the currency the NUMBER is written in "
            "goes in amount_currency_code, the currency of the ANSWER goes "
            "in result_currency_code.")

    if rate_date is not None:
        if not isinstance(rate_date, str) or not _RE_DATE.match(rate_date):
            _fail("rate_date", rate_date,
                "expected an ISO 8601 date formatted YYYY-MM-DD, or None.",
                "Resolve relative phrases yourself: 'the first of August' "
                "in 2026 -> '2026-08-01'.")

    rate = _FX[result_currency_code] / _FX[amount_currency_code]
    return {"amount": round(amount * rate, 2),
        "currency": result_currency_code, "rate": round(rate, 6),
        "as_of": rate_date or "latest"}

```

```

# -----
# Tool 4
# -----
def send_notification(recipient_address, channel, message_body,
    priority="normal"):
    """Send a one-off notification to a single person.

    Parameters
    -----
    channel : str
        CLOSED SET – exactly one of: "email", "sms", "slack", "push".
        Map the user's verb: "text"/"SMS" -> "sms", "message"/"DM"/"ping on
        Slack" -> "slack", "notify on their phone" -> "push".
    recipient_address : str
        The address, in the format REQUIRED BY `channel`. The formats are not
        interchangeable – an email address is not a valid SMS recipient.
        channel="email" -> name@domain.tld
            e.g. "marco.silva@example.com"
        channel="sms" -> E.164 phone number, "+" then country code
            then digits, e.g. "+14155550142"
        channel="slack" -> "@handle" or a member id like "U04AB12CD"
        channel="push" -> a 32+ character device token
    You will often hold only one identifier for a person (usually their
    email). That does NOT make it usable on another channel: call
    `lookup_contact(person, channel)` to get the right address, or ask the
    user. Do not pass an email address to a non-email channel.
    message_body : str
        Plain text, 1-1000 characters. No markup, no attachments.
    priority : str
        CLOSED SET – exactly one of: "low", "normal", "high". Default "normal".
        Map the user's wording:
        urgent / ASAP / critical / P0 / emergency / important -> "high"
        medium / standard / (unstated) -> "normal"
        FYI / whenever / no rush -> "low"
        There is no level above "high"; "urgent" is "high".

    Raises
    -----
    ValueError
        If the channel or priority is not in its set, or the address does not
        match the format that channel requires.
    """
    channel = _closed_set("channel", channel, _CHANNELS, {
        "text": "sms", "txt": "sms", "message": "slack", "dm": "slack",

```

```

    "mail": "email", "e-mail": "email", "notification": "push"})
priority = _closed_set("priority", priority, _PRIORITIES,
                        _PRIORITY_SYNONYMS)

spec = {
    "email": (_RE_EMAIL, "an email address of the form name@domain.tld",
              "marco.silva@example.com"),
    "sms": (_RE_E164, "an E.164 phone number starting with '+'",
            "+14155550142"),
    "slack": (_RE_SLACK, "a Slack handle '@name' or member id 'U04AB12CD'",
              "@dana"),
    "push": (_RE_PUSH, "a device token of at least 32 characters",
             "d9f1c0a4e77b4e2b9a3c5d6e7f8091a2"),
}

[channel]
pattern, human, example = spec
if not isinstance(recipient_address, str) or \
    not pattern.match(recipient_address.strip()):
    extra = ""
    if _RE_EMAIL.match(str(recipient_address or "")) and channel != "email":
        extra = (f" You passed an email address to channel='{channel}'. "
                 f"Call lookup_contact(person, '{channel}') to get their "
                 f"{channel} address, or send it by email instead.")
    _fail("recipient_address", recipient_address,
          f"is not valid for channel='{channel}', which requires {human}.",
          f"Example: {example!r}. {extra}")

if not isinstance(message_body, str) or not message_body.strip():
    _fail("message_body", message_body, "expected non-empty plain text.",
          "Write the message the user asked to send.")
if len(message_body) > 1000:
    _fail("message_body", f"<{len(message_body)} characters>",
          "exceeds the 1000 character limit.",
          "Shorten the message or send it by email as a document.")

return {"message_id": "msg_0001", "channel": channel,
        "to": recipient_address.strip(), "priority": priority,
        "chars": len(message_body)}

```

```

# -----
# Tool 5
# -----
def resize_image(source_path, target_path, width_px=None, height_px=None,
                 preserve_aspect_ratio=True, jpeg_quality=85):
    """Resize an image file and write the result to a new path.

    This tool speaks ABSOLUTE PIXELS ONLY. It cannot scale by a percentage or
    a factor, and it does not know the source image's current size. If the
    user asks for something relative ("half the size", "25% smaller", "make
    it a bit bigger"), call `get_image_info(source_path)` first to read the
    current width and height, do the arithmetic yourself, and pass the
    resulting absolute pixel values. Never pass 50 to mean "50%" and never
    pass 0.5 to mean "half".

    Parameters
    -----
    source_path : str
        Path of the image to read. Must be a non-empty string.
    target_path : str
        Path to write the resized image to. Must differ from source_path
        unless the user explicitly asked to overwrite the original.
    width_px : int or None
        Target width in WHOLE PIXELS (e.g. 800), or None to let the width be
        derived from height_px. At least one of width_px / height_px must be
        an integer.
    height_px : int or None
        Target height in WHOLE PIXELS, or None to let the height be derived
        from width_px.
    "Resize logo.png to 800 pixels wide" -> width_px=800, height_px=None.
    Do NOT repeat the width as the height: width_px=800, height_px=800
    means "fit inside an 800x800 box", which is a different request.

```

```

preserve_aspect_ratio : bool
    True (default) fits the image inside the given box without distortion,
    so the output may be smaller than the numbers you passed on one axis.
    False stretches the image to exactly width_px x height_px, which
    requires BOTH to be integers. The word "exactly" in a user's request
    means False.
jpeg_quality : int
    Encoder quality 1-100, default 85. Only meaningful when target_path
    ends in .jpg/.jpeg; ignored otherwise. "high quality" -> 95.

Raises
-----
ValueError
    If both dimensions are None, if a dimension looks like a percentage or
    a scale factor, or if an exact resize is requested with a missing axis.
"""
for name, val in (("source_path", source_path), ("target_path", target_path)):
    if not isinstance(val, str) or not val.strip():
        _fail(name, val, "expected a non-empty file path string.",
              "Example: 'logo.png'.")

for name, val in (("width_px", width_px), ("height_px", height_px)):
    if val is None:
        continue
    if isinstance(val, bool):
        _fail(name, val, "expected a pixel count or None.", "Example: 800.")
    if isinstance(val, float) and 0 < val < 1:
        _fail(name, val,
              "looks like a scale factor rather than a pixel count.",
              "This tool takes absolute pixels only. Call "
              "`get_image_info(source_path)`, multiply the real dimension "
              "yourself, and pass the result, e.g. 960.")
    if not isinstance(val, int):
        _fail(name, val, "expected a whole number of pixels or None.",
              "Example: 800. Do not pass '800px' or 800.5.")
    if val < 1 or val > 20000:
        _fail(name, val, "is outside the supported range of 1-20000 pixels.",
              "Check whether this was meant as a percentage; if so, read "
              "the real size with `get_image_info` and convert it.")

if width_px is None and height_px is None:
    _fail("width_px/height_px", (width_px, height_px),
          "at least one dimension must be given.",
          "Pass width_px for 'N pixels wide', height_px for 'N pixels "
          "tall', or both for a bounding box. For a relative request such "
          "as 'half the size', read the current size with "
          "`get_image_info(source_path)` and compute the pixels first.")

if not isinstance(preserve_aspect_ratio, bool):
    _fail("preserve_aspect_ratio", preserve_aspect_ratio,
          "expected True or False.",
          "True fits inside the box; False stretches to exact dimensions.")
if preserve_aspect_ratio is False and (width_px is None or height_px is None):
    _fail("preserve_aspect_ratio", preserve_aspect_ratio,
          "an exact (non-proportional) resize needs both width_px and "
          "height_px.",
          "Either supply the missing dimension, or set "
          "preserve_aspect_ratio=True to scale from the one you have.")

if isinstance(jpeg_quality, bool) or not isinstance(jpeg_quality, int):
    _fail("jpeg_quality", jpeg_quality, "expected a whole number 1-100.",
          "Default is 85; use 95 for 'high quality'.")
if not 1 <= jpeg_quality <= 100:
    _fail("jpeg_quality", jpeg_quality, "is outside the range 1-100.",
          "Use 85 for the default or 95 for high quality.")

return {"path": target_path, "width": width_px, "height": height_px,
        "quality": jpeg_quality, "fit": preserve_aspect_ratio}

```

TOOLS = {

```
"schedule_meeting": schedule_meeting,  
"search_orders": search_orders,  
"convert_currency": convert_currency,  
"send_notification": send_notification,  
"resize_image": resize_image,  
}
```

Appendix B — how to run this yourself

Everything is plain Python with no dependencies. Unzip the files into a folder and run:

```
python3 run_round1.py    # the original tools: 6 right, 9 wrong
python3 run_round2.py    # the fixed tools: 15 right, plus all 18 checks
```

What each file is for:

FILE	WHAT IT HOLDS
<code>tools_v1.py</code>	The five original tools. Good names, detailed docstrings, no input checks.
<code>tools_v2.py</code>	The same five, rewritten. New docstrings and full input checking.
<code>cases.py</code>	The 15 requests, the arguments the model sent, and the arguments a correct answer needed.
<code>cases_v2.py</code>	The retest, plus the 18 deliberately broken calls used to test the error messages.
<code>run_round1.py</code>	Runs the first round and writes <code>round1_results.json</code> .
<code>run_round2.py</code>	Runs the retest and the check suite, and writes <code>round2_results.json</code> .

To try it with your own tools, replace the five in `tools_v1.py`, write three natural phrasings per tool in `cases.py`, and record what your model sends back.